

Minimality Metric Implementation in the Metis Framework

Noel Bastubbe

Hasso Plattner Institute

Potsdam, Germany

noel.bastubbe@student.hpi.de

1 Introduction

This report documents the implementation of a minimality metric within the Metis framework [10], developed as part of the Data Quality Application course. It begins by defining the data quality dimension of minimality and presenting the measurement function, including its inputs, outputs, and integration into Metis. This is followed by a discussion of the metric’s aggregation behavior, soundness, and scalability, as well as the customization options it exposes to the user. The report then discusses how the metric should be presented to the user in a transparent and actionable way. Subsequently, the two implemented similarity approaches — a *type-aware* clustering method and a SemHash-based semantic approach — are evaluated against each other on five publicly available benchmark datasets, comparing their results to ground truth minimality scores across a range of similarity thresholds. It concludes with an outlook on potential extensions and directions for future work.

2 Minimality as a Data Quality Dimension

Minimality is a data quality dimension that focuses on whether data is represented as concisely as possible, capturing the relevant aspects of the domain of interest without unnecessary excess. The dimension is also referred to in the literature under the synonymous terms non-redundancy, uniqueness, compactness, and conciseness. Batini & Scannapieco [3] define it as “the capability of representing the aspects of the reality of interest with the minimal use of informative resources.”

In practice, a dataset fails to meet the minimality criterion when it contains “too much” data. This excess manifests in two forms: (1) redundant data and (2) irrelevant data.

(1) Redundant data refers to information that is represented more than once. This may occur as duplicate tuples, where multiple rows correspond to the same real-world entity, or as semantically redundant values within a column, where different values convey the same meaning. Additionally, columns containing information that can be derived from other attributes—such as age from a birth date—are also considered redundant.

(2) Irrelevant data, in contrast, consists of information that is not necessary for the intended task. This may include outdated data or data that does not provide meaningful value for the domain being modeled. Both forms of excess are undesirable, as they increase storage requirements, complicate data processing, and hinder the extraction of meaningful insights [3].

3 The Minimality Metric

The minimality metric implemented in this report is defined at the table level and focuses exclusively on the (1) redundancy aspect of minimality. It operates under the assumption that identical records

indicate redundancy and reduce the dataset’s minimality. This assumption does not always hold in practice, as identical records may occasionally correspond to distinct real-world entities; however, such cases are not distinguished by the metric and are treated as duplicates. The detection of irrelevant data is deliberately excluded from this metric, as determining whether data is irrelevant requires domain expertise, making it significantly harder to automate in a generalized metric.

The metric is adapted from the schema-level metric proposed by Ehrlinger & Wöß [6], who define minimality as a three-step process: similarity calculation, clustering, and minimality calculation. In the original formulation, schema elements are compared pairwise using a similarity function $\sigma : e \times e \rightarrow \mathbb{R} \in [0, 1]$ [7], where 1.0 indicates a perfect match and 0.0 indicates complete dissimilarity. Elements are then grouped into clusters based on their similarity, and the minimality score is calculated as:

$$Min(s) = \frac{|c| - 1}{|e| - 1} \quad (1)$$

where $|c|$ is the number of clusters and $|e|$ is the total number of elements. The subtraction of 1 in both the numerator and denominator serves as a normalization factor, ensuring that the metric is bounded in $[0, 1]$ and yields a value of 1.0 in the ideal case where each element forms its own cluster (i.e., $|c| = |e|$). By definition, the score is also set to 1.0 when $|e| = 1$, avoiding the undefined case of division by zero and reflecting that a single-element dataset is trivially minimal. A perfectly minimal dataset therefore achieves a score of 1.0, while the presence of duplicate or near-duplicate elements reduces the number of clusters and lowers the score accordingly.

At the table level, each row of the dataset is treated as an element. The similarity function is adapted accordingly to measure the closeness between two rows rather than two schema elements, while preserving the same definition and value range. Rows that are sufficiently similar are grouped into the same cluster, and the minimality score is calculated using the same formula as above.

4 Input and Output

The metric takes four inputs. The first is the dataset to be assessed. The second is a similarity function following the definition introduced above, which must be defined for all pairs of rows within the dataset and returns a value in $[0, 1]$ indicating how closely two rows resemble each other. The third input is a similarity threshold, also in the range $[0, 1]$, which determines when two rows are considered similar enough to belong to the same cluster. Any pair of rows whose similarity meets or exceeds this threshold will be treated as duplicates. The fourth input is a cluster linkage strategy, which defines how clusters are built from pairwise similarity values, for example by using the minimum, maximum, or average similarity

between members of two clusters to decide whether they should be merged.

The output of the metric is a single score in the range $[0, 1]$. A score of 1.0 indicates that no duplicate records were detected, meaning the dataset is fully minimal with respect to redundancy. A score of 0.0 indicates that all records in the dataset were found to be duplicates of one another. Values in between reflect the degree to which redundancy is present in the data.

5 Implementation

The metric is implemented in the Metis framework [10] and currently accepts three of the four inputs defined above. Based on findings by Ehrlinger & Wöß [6], who observed no significant impact of the linkage strategy on the metric output at schema level, the implementation fixes the clustering strategy to single linkage. Hierarchical clustering is performed using the linkage and fcluster functions from the `scipy.cluster.hierarchy` library [14].

The metric can be applied to any tabular dataset. The similarity threshold is configurable and defaults to 0.85 if not specified by the user.

The user can choose between two similarity functions. The first is a semantic hashing approach using the *SemHash* library [11], which computes similarity based on the semantic meaning of rows. It should be noted that this approach only considers columns with an object dtype, meaning non-text columns are ignored entirely when this backend is selected. The second is a *type-aware* similarity function implemented specifically for this metric. It explicitly filters for supported column types — strings, categories, numbers, booleans, and timestamps — with columns outside these types being silently dropped. Rather than treating all remaining column values as strings, it selects an appropriate similarity measure based on the data type of each column. String columns are compared using normalized Levenshtein similarity [2], which accounts for differences in character edit distance relative to string length. Numeric columns are compared based on their normalized absolute difference. Timestamps are compared by their relative difference in seconds, also normalized. Categorical and boolean columns are treated as either identical or completely dissimilar, returning 1.0 for a match and 0.0 otherwise. NaN values in either row are skipped during comparison. The final row similarity is computed as the mean of the individual column similarities. All data manipulation is handled using the pandas library [12].

6 Aggregation

The metric is primarily designed to be calculated at the table level, which is also the granularity at which it is implemented in Metis. While the original metric by Ehrlinger & Wöß [6] is defined at the schema level, the adaptation to the table level follows the same underlying principle by treating rows as elements.

In theory, the metric could also be applied at the column level, where any repeated value would be treated as a data quality issue. This could be meaningful for columns that are expected to contain unique values, or for combinations of columns that together should be unique. However, column-level evaluation quickly loses meaning for columns where repetition is expected and valid. Evaluating minimality on a boolean column, for instance, is not useful since

it can by definition only contain two distinct values. Furthermore, aggregating from column-level scores to a table-level score is not appropriate. Consider a table with four boolean columns and 16 rows where every row is distinct: at the table level the minimality score would correctly be 1.0, as each row represents a unique entity. At the column level, however, each boolean column would receive a score of $\frac{1}{15}$, and averaging these would produce a misleading result that contradicts the table-level assessment.

Calculating the metric at the cell level is not meaningful, as each cell contains a single value whose similarity with itself is always 1.0 by definition.

Aggregation upward from table level to database level is possible by computing the mean of the table-level scores, provided that there are no duplicate records across tables. If cross-table duplicates are present, this aggregation would overestimate the minimality of the database as a whole.

7 Soundness

To evaluate whether the metric is sound, the five requirements for data quality metrics defined by Heinrich et al. [8] are applied.

R1 – Existence of Minimum and Maximum Metric Values.

The metric is bounded by $[0, 1]$, where 1.0 represents perfect minimality and 0.0 indicates that all records in the dataset are duplicates of one another. This bounding allows metric values to be compared over time and makes it possible to observe improvements in data quality. The metric fulfills R1.

R2 – Interval-Scaled Metric Values. The metric achieves a ratio scale, which is stricter than a simple interval scale. In addition to equally spaced units, a ratio scale has a defined zero point and preserves ratios between values, meaning it is possible to make statements such as “dataset A is twice as redundant as dataset B.” The metric fulfills R2.

R3 – Quality of the Configuration Parameters and Determination of the Metric Values. All configuration parameters of the metric can be determined automatically. The number of rows and clusters can be counted objectively, and the similarity between rows is computed deterministically by the chosen similarity function. The metric therefore fulfills the criteria of objectivity, reliability, and validity required by R3.

R4 – Sound Aggregation of the Metric Values. R4 requires that a metric produce consistent results across all levels of aggregation [8]. The metric does not fully satisfy this requirement. The metric is not meaningfully defined at the tuple level, as a single row cannot be assessed for redundancy in isolation. At the relation level, the metric is well-defined and produces consistent results. Aggregation to the database level is possible by computing the mean across tables, but only under the condition that no duplicate records exist across tables — if cross-table duplicates are present, the aggregated score would overestimate the overall minimality of the database. R4 is therefore partially fulfilled.

R5 – Economic Efficiency of the Metric. All configuration parameters can be determined automatically without requiring manual effort or domain expertise, meaning the cost of applying the metric is low relative to the insight it provides. It should be noted that designing and selecting a suitable similarity function tailored to a specific domain can add to the overall cost. However,

this is not required for basic application, as the metric ships with a generalized similarity function that can be used out of the box. The metric fulfills R5.

In summary, the table-level minimality metric fulfills four of the five requirements defined by Heinrich et al. [8] fully, and partially fulfills R4. It can therefore be considered a sound data quality metric.

8 Scalability

The current implementation does not scale well to large datasets. This is an inherent challenge of duplicate detection in general, as the number of possible row pairs grows quadratically with the number of records [4]. The *type-aware* similarity backend performs a naïve pairwise comparison of all records using a nested loop, which computes similarity for every possible pair exactly once. This results in $O(n^2)$ comparisons and quickly becomes computationally infeasible for large tables [4].

A common approach to address this is blocking [4], which reduces the number of candidate pairs to a manageable size by grouping records into blocks and only comparing records within the same block. The key trade-off when applying blocking is that it prioritizes recall over precision — the goal is to avoid missing true duplicates rather than to eliminate all false candidates. Several blocking strategies exist. Standard blocking hashes records on a blocking key, such as the first two digits of a zip code or the first letter of a surname, and only compares records that share the same hash value. Sorted neighborhood methods [9] sort records by a key and compare only those that fall within a sliding window of a fixed size. Other blocking methods exist beyond these two.

Incorporating a blocking step into the implementation would be a meaningful improvement for handling large datasets and is a natural candidate for future work.

9 Customization

The minimality metric is highly customizable in principle. The similarity function, clustering linkage strategy, and similarity threshold can all be tailored to a specific use case, allowing the metric to be adapted to the characteristics of the data and the domain requirements. For example, a domain-specific similarity function could be designed to weight certain columns more heavily, or to handle domain-specific data formats more accurately than a generalized function.

The current implementation in Metis [10] offers limited customization in comparison. The similarity threshold is freely configurable and defaults to 0.85. However, the choice of similarity function is restricted to two options — the SemHash-based semantic approach and the *type-aware* similarity function — and the clustering linkage strategy is fixed to single linkage. Extending the implementation to support additional similarity functions and linkage strategies would increase its flexibility and is discussed further in the outlook.

10 Experiments

To evaluate the two implemented approaches, both the *type-aware* clustering method and the SemHash-based method were run on all five datasets across similarity thresholds ranging from 0.10 to

Table 1: Dataset characteristics and ground truth minimality scores.

Dataset	Tuples	Clusters	GT Score	Coverage
2MASS	400	200	0.4987	Partial (7.5%)
Drives	835	96	0.1139	Exhaustive
Notebooks1	343	60	0.1725	Exhaustive
Notebooks2	337	85	0.2500	Exhaustive
Restaurant	864	752	0.8702	Exhaustive

1.00 in steps of 0.05. The results were compared against the ground truth minimality scores established for each dataset.

All five datasets are publicly available benchmark datasets for duplicate detection. Notebooks1 and Notebooks2 correspond to datasets X2 and X3 from the ACM SIGMOD 2021 Programming Contest on entity resolution [1], where the entities represent laptop products listed across different e-commerce websites. Drives corresponds to dataset X4 from the same contest [1], containing records of storage devices — primarily pen drives and flash drives — sourced from the altosight.com platform. Restaurant is a well-known duplicate detection benchmark distributed as part of the DuDe duplicate detection toolkit [5], containing restaurant records with structured text columns for name, address, city, phone number, and cuisine type. 2MASS is derived from the Two Micron All Sky Survey astronomical catalog [13] and, unlike the other four datasets, contains no natural-language text columns, consisting entirely of numeric measurements such as coordinates, observation dates, and infrared magnitude readings.

For four of the datasets — 2MASS, Drives, Notebooks1, and Notebooks2 — the ground truth was originally provided in the form of query-answer pairs, which were converted into cluster assignments for use with this metric. For the Restaurant dataset, the ground truth was directly available as duplicate pairs. It should be noted that the 2MASS ground truth covers only 7.5% of all possible pairs, meaning its ground truth score is an approximation rather than an exhaustive labeling of all duplicates. All other datasets have exhaustive ground truth coverage.

The ground truth scores and dataset characteristics are summarized in Table 1.

To identify the best-performing threshold for each method, the mean absolute error between the metric output and the ground truth score was computed across all datasets for each threshold. For the *type-aware* clustering method, the threshold of 0.75 produced the lowest mean absolute error of 0.0757. For SemHash, the optimal threshold was 0.85, yielding a mean absolute error of 0.1361. The per-dataset scores at these optimal thresholds are shown in Tables 2 and 3.

At their respective optimal thresholds, the *type-aware* clustering method achieves a lower mean absolute error across all datasets combined. However, SemHash performs more consistently on four of the five datasets, with 2MASS being a notable outlier where it assigns a score of 1.0 despite a ground truth of approximately 0.50. This is a direct consequence of SemHash operating exclusively on text columns — since 2MASS contains no natural-language data, SemHash finds no duplicates at all. The *type-aware* method handles

Table 2: Type-aware clustering results at threshold 0.75.

Dataset	Metric Score	Ground Truth	Abs. Error
2MASS	0.5063	0.4987	0.0076
Drives	0.0252	0.1139	0.0887
Notebooks1	0.0789	0.1725	0.0936
Notebooks2	0.3214	0.2500	0.0714
Restaurant	0.9873	0.8702	0.1171
Mean			0.0757

Table 3: SemHash results at threshold 0.85.

Dataset	Metric Score	Ground Truth	Abs. Error
2MASS	1.0000	0.4987	0.5013
Drives	0.0779	0.1139	0.0360
Notebooks1	0.0994	0.1725	0.0731
Notebooks2	0.2411	0.2500	0.0089
Restaurant	0.9316	0.8702	0.0614
Mean			0.1361

2MASS well precisely because it is designed to compare numeric columns directly. On the four text-based datasets, SemHash generally performs competitively and achieves smaller errors than the *type-aware* method on Drives, Notebooks2, and Restaurant at their respective optimal thresholds.

11 Presentation and Explanation

For the metric to be useful in practice, the output needs to be presented to the user in a way that is transparent and actionable. At a minimum, the similarity function used and the similarity threshold should always be displayed alongside the score, as the score has little meaning without knowing the configuration it was computed under. A small change in the threshold can significantly alter the number of detected duplicates and therefore the final score.

Beyond the score itself, it is valuable to annotate each row with the cluster ID it was assigned to. This allows users to quickly identify which groups of records were considered duplicates, making the output directly useful for data cleaning purposes. Currently, the Metis implementation [10] only outputs the total number of clusters and the final minimality score, without exposing cluster assignments. Surfacing this information would be a meaningful improvement to the usability of the metric.

12 Outlook and Future Work

The current implementation provides a functional baseline for measuring minimality at the table level, but several extensions could improve its flexibility, scalability, and usefulness in practice.

The most impactful improvement in terms of usability would be to make the metric more plug-and-play with respect to its core components. Currently, the similarity function is limited to two built-in options and the linkage strategy is fixed to single linkage. Both could be extended in two complementary ways: by offering a broader selection of options out of the box, and by allowing users to

inject custom similarity functions or linkage strategies directly into the metric calculation. This would make the metric significantly more adaptable to domain-specific use cases without requiring changes to the implementation itself.

Scalability is a further area for improvement. The current *type-aware* backend performs a naïve pairwise comparison of all records, which becomes computationally infeasible for large datasets [4]. Introducing a blocking step — for example using standard blocking [4] or sorted neighborhood methods [9] — would reduce the number of candidate pairs to a manageable size and allow the metric to be applied to much larger tables.

Finally, the output of the metric could be made more actionable for data cleaning purposes. At present, only the total number of clusters and the final minimality score are returned. Exposing the cluster assignment for each row would allow users to directly identify which records were considered duplicates, making it straightforward to review and resolve them. Extending the metric to detect duplicates across tables rather than within a single table would additionally enable more meaningful aggregation at the database level.

References

- [1] 2021. ACM SIGMOD 2021 Programming Contest. <https://dbgroup.ing.unimo.it/sigmod21contest/>.
- [2] Max Bachmann. 2023. python-Levenshtein: Fast String Matching in Python. <https://github.com/maxbachmann/Levenshtein>.
- [3] Carlo Batini and Monica Scannapieco. 2016. *Data and Information Quality: Concepts, Methodologies and Techniques*. Springer, Cham. doi:10.1007/978-3-319-24106-7
- [4] Peter Christen. 2012. *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Springer, Berlin, Heidelberg. doi:10.1007/978-3-642-31164-2
- [5] Uwe Draisbach and Felix Naumann. 2010. DuDe: The Duplicate Detection Toolkit. In *Proceedings of the International Workshop on Quality in Databases (QDB)*. Singapore. <https://hpi.de/naumann/projects/data-integration-data-quality-and-data-cleansing/dude.html>
- [6] Lisa Ehrlinger and Wolfram Wöß. 2019. A Novel Data Quality Metric for Minimality. In *Proceedings of the International Workshop on Quality in Databases (QUAT 2018) (Lecture Notes in Computer Science, Vol. 11235)*. Springer, 1–15. doi:10.1007/978-3-030-19143-6_1
- [7] Jérôme Euzenat and Pavel Shvaiko. 2007. *Ontology Matching*. Springer, Secaucus, NJ. doi:10.1007/978-3-540-49612-0
- [8] Bernd Heinrich, Diana Hristova, Mathias Klier, Alexander Schiller, and Michael Szubartowicz. 2018. Requirements for Data Quality Metrics. *Journal of Data and Information Quality* 9, 2 (2018), 12:1–12:32. doi:10.1145/3148238
- [9] Mauricio A. Hernández and Salvatore J. Stolfo. 1995. The Merge/Purge Problem for Large Databases. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. ACM, San Jose, CA, 127–138.
- [10] HPI Information Systems Group. 2024. Metis: A Framework to Automatically Assess the Quality of Tabular Data across Multiple Dimensions. <https://github.com/HPI-Information-Systems/Metis>.
- [11] MinishLab. 2024. SemHash: Semantic Hashing for Fast and Lightweight Text Deduplication. <https://github.com/MinishLab/semhash>.
- [12] The Pandas Development Team. 2024. *pandas: Powerful Python Data Analysis Toolkit*. doi:10.5281/zenodo.3509134
- [13] University of Massachusetts and Infrared Processing and Analysis Center. 2003. Two Micron All Sky Survey (2MASS). <https://www.ipac.caltech.edu/2mass/>.
- [14] Pauli Virtanen et al. 2020. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods* 17 (2020), 261–272. doi:10.1038/s41592-019-0686-2